

Preparació de l'Examen de Verificació PRO1

14 de desembre del 2022

Exercici 1. Senars.

Verifiqueu aquesta funció:

```
1 int nombreSenars (const list<int> &l)
2 // Pre : cert
3 {
4     int r = 0;
5
6     iterador it2 = l.begin();
7
8     while (it2 != l.end())
9     {
10         if (*it2 % 2 == 1) r = r + 1;
11         ++it2;
12     }
13
14     return r;
15 // Post: r = | { *it ∈ l | *it mod 2 = 1 } |
16 };
```

Com podeu veure, la postcondició es pot llegir d'aquesta manera: *r té la cardinalitat del conjunt d'elements *it (és a dir, valors de la llista l) tals que compleixen la condició de ser senars.* Dit altrament: *r té quants nombres senars conté la llista l.*

Solució

Primer cal trobar un invariant. Com en aquests casos acostuma a succeir, agafarem una debilitament de la postcondició:

$$r = | \{ *it \in l \mid *it \bmod 2 = 1 \wedge l.begin \leq it < it2 \} |$$

Aquest invariant ens diu que portem calculada la solució (és a dir, el nombre de valors senars de la llista *l*) fins a la posició *it2*. Com ja podeu intuir, quan *it2 = l.end()* voldrà dir que haurem calculat el resultat que ens demana la postcondició.

Ara podem començar la verificació.

1. Precondició + inicialitzacions $\Rightarrow$ INV.

En aquest cas, cal verificar que es compleix l'invariant, tenint en compte que el primer que fem és l'assignació *it2 = l.begin()*; i també *r = 0*; llavors l'invariant queda així (substituint les variables assignades):

$$0 = | \{ *it \in l \mid *it \bmod 2 = 1 \wedge l.begin \leq it < it2 \} |$$

Tenint en compte que l'interval *l.begin ≤ it < l.begin()* és buit, la part dreta de la igualtat equival al neutre de l'operador suma que és 0.

2. Condició del bucle + INV $\Rightarrow$ INV.

Cal veure que si l'invariant és cert **abans** d'entrar al bucle, després d'executar les instruccions de dins del bucle, l'invariant seguirà sent cert. Fixem-nos que les instruccions de dins del bucle són només

```
if (*it2 % 2 == 1)  r = r + 1;
++it2;
```

Per tant, l'invariant **després** de les instruccions del bucle serà aquest:

$$r = | \{ *it \in l \mid *it \bmod 2 = 1 \wedge l.begin \leq it < it2 + 1 \} |$$

ja que hem fet `++it2`; La part de la dreta es pot descomposar en dues parts: la que ja portàvem calculada i la que hem calcula a dins del bucle, i això és el que caldrà veure si és cert:

$$r = | \{ *it \in l \mid *it \bmod 2 = 1 \wedge l.begin \leq it < it2 \} \cup \\ \{ *it \in l \mid *it \bmod 2 = 1 \wedge it2 \leq it < it2 + 1 \} |$$

Ara bé, fixem-nos que la segona part ($it2 \leq it < it2+1$) només un element: `*it2`. Per tant, ara cal verificar si és cert un invariant que ens diu que r té el comptatge de tots els senars entre $l.begin \leq it < it2$ i els que hi ha a `*it2`. Això és fàcil de veure que és cert perquè la primera part ja ho garantia l'invariant **abans** de fer la volta del bucle, i la segona part ho garanteix la instrucció `if (*it2 % 2 == 1) r = r + 1;`, ja que si `*it2` és senar, sumem 1 a r , i si no ho és, no fem res.

Finalment cal tenir en compte que les operacions dins del cos del bucle que no són totals (`++it2` o l'accés a `*it2`) estan protegides per la condició del bucle.

3. Negació de la condició + $INV \Rightarrow POST$. Quan acabem tenim que es compleix la negació de la condició: $it2 == l.end()$. Si substituïm $it2$ per $l.end()$ a l'invariant tenim

$$r = | \{ *it \in l \mid *it \bmod 2 = 1 \wedge l.begin \leq it < l.end() \} |$$

que és, precisament, la postcondició de la funció.

4. Funció fita. Com a funció fita, en aquests casos, agafarem, de manera genèrica, la mida de la part de l'estructura que ens queda per tractar. En aquest cas, seria la funció tal que, donat l'iterador `it2` ens torna el nombre d'elements que hi ha entre `it2` i `l.end()`. Com que és una mica, llavors està definida en els naturals A més, a cada pas de la iteració executem `++it2`, la qual cosa fa que el valor d'aquesta funció decreixi estrictament a

cada pas de l'execució. Tinguem en compte que `++it2` o l'accés a `*it2` no són funcions totals (no estan definides quan `it2` és igual a `1.end()`), però precisament això no pot passar perquè estem protegits per la condició del bucle.

Exercici 2. Arbre de sumes.

Verifiqueu aquesta funció:

```
1  int arbreSumes(const BT &t, BT &h)
2  // Pre: cert.
3  {
4      int r = 0;
5
6      if (t.isEmpty()) {
7
8          h = BT();
9          return r;
10     }
11     else {
12
13         BT fe = BT();
14         BF fd = BT();
15
16         r = t.getRoot() +
17             arbreSumes(t.getLeft(), fe) +
18             arbreSumes(t.getRight(), fd);
19
20         h = BT(r, fe, fd);
21     }
22
23     return r;
24 }
25 // Post: h es l'arbre de sumes de l'arbre t
26         i r es la suma de l'arbre t.
```

Un **arbre de sumes** d'un arbre **T** és un arbre amb la mateixa estructura, i a on cada posició conté la suma de nodes del subarbre que penja d'aquella mateixa posició a l'arbre **T**.

Solució

En primer lloc, cal veure que hi ha un sol paràmetre (**t**) i dos valors de tornada: l'arbre **h** que es retorna per referència i l'enter **r** que es retorna per valor.

El cas recursiu és fàcil: si l'arbre **t** és buit, el resultat només pot ser un arbre buit i un zero (la suma d'un arbre buit és el neutre de la suma).

Per al cas recursiu cal verificar que:

1. Es compleix la precondition de les crides recursives. En aquest cas, com que la pre és trivial (**cert**) no cal comprovar res perquè es compleix de manera trivial.
2. Les operacions que executem en el cas recursiu es poden executar totes. Cal que ens fixem en les operacions no totals, en aquest cas són **getRoot**, **getLeft** i **getRight**, que només es poden executar sobre arbres no buits. Precisament, aquestes instruccions estan sota el cas **else**, és a dir, que **no són buits**, i, per tant, totes les operacions es poden executar.
3. Cal verificar que després de les crides recursives, arribem a la post. En aquest cas, com que assumim com a HI que les crides recursives s'executen

correctament, tenim que r contindrà la suma del fill dret, la del fill esquerre i també el de l'arrel de l'arbre t , és a dir, la suma de tot l'arbre t , cosa que fa complir una part de la potcondició, concretament, la part r és la suma de l'arbre t .

L'altra part (h és l'arbre suma de l'arbre t) es compleix perquè després de les crides recursives (seguim assumint que les crides recursives s'han executat correctament) tenim que fe i fd tenen els arbres suma dels fills esquerre i dret de l'arbre t . Com que h és l'arrelament de r , fe i fd , tenim que h és l'arbre suma (per definició).

4. Definim una funció de fita. En aquest cas, una vegada més, és l'alçada de l'arbre. El raonament el teniu en el document amb exercicis resolts. Aquí cal que tingueu en compte que el decreixement es compleix **en totes dues crides**. Dit altrament: el raonament que fem sobre el decreixement de la fita cal aplicar-lo a totes dues crides recursives. Això no vol dir dues demostracions, sinó una sola, en què cal dir, però, que s'aplica a totes dues crides recursives. Tingueu en compte que si només podem verificar que una sola crida recursiva fa decreixer la fita, llavors no podríem tenir la certesa que la recursivitat acaba.